



PROJECT REPORT

Smart spending, student style.

A budget and expense tracker built for an allowance that lands when it lands — not for a salary and a bank feed.

THEME

NextGen BudgetBee

VERSION

1.0

LIVE SITE

campuscoin-sable.vercel.app

CATEGORY

End-to-End Web Solutions

STACK

MongoDB · Express · React · Node

SOURCE

github.com/hamzaharoonk07-cloud/campuscoin

CONTENTS

What is in this report

01	The problem, and the shape of the answer	
02	Installation	Requirements, first run, development, tests
03	User credentials	Every role, and how the two doors differ
04	Passwords and sessions	Hashing, recovery, lockout, revocation
05	Configuration	Every variable, and the behaviour without it
06	Technology	
07	How the application is put together	Layout and the engines
08	Diagrams and database design	Context, DFD, two flowcharts, the ER diagram, every collection's fields
09	The parts that make judgements	Where a model is used, and where it is not
10	Design and accessibility	
11	Deploying to Vercel	
12	Deliverables checklist	
13	The interface	Screens, in order of use

01

The problem, and the shape of the answer

Students are paid irregularly — an allowance when it arrives, a bit of tutoring money, a scholarship instalment — and general personal-finance apps assume a salary and a bank feed. Every one of them is either built for someone else's money or asks for a bank login a student does not have. Campus Coin assumes neither: no linked account, no subscription, nothing to connect — just a fast way to log what actually happened and advice built from that student's own numbers, not a stranger's average.

Everything is entered by hand or imported from a CSV, and every piece of advice the application gives is built from the student's own transactions. Nothing is generic: a saving tip quotes real figures from that student's history, and the chat assistant answers from the same aggregations the reports are drawn from, so the two can never disagree.

RUNS ON A FRESH MACHINE

A MongoDB server is *optional*. With no connection string configured the application starts its own embedded MongoDB and stores it in `server/data/db`, so it installs and demonstrates every feature with an empty configuration file.

AI TOOLS USED, ACKNOWLEDGED

Built with **Claude Code** (Anthropic) as a coding and documentation assistant throughout — implementation, debugging, this report's diagrams and build scripts, and its PDF-export tooling. It was used as an assistant, not a template generator: no part of the application was produced from a ready-made site or theme, and every design and architecture decision in this document is one its author can explain and defend. The application's own optional AI features (the categorisation assistant and Coin's monthly insight, section 09) call the **Claude API** directly and are a separate, documented part of the product itself, not a documentation shortcut.



02

Installation

Requirements: Node.js 18 or newer.

```
# 1. Install both halves
npm run setup

# 2. Copy the environment template (all values are optional for a local run)
cp server/.env.example server/.env

# 3. Build the React front end
npm run build

# 4. Start it
npm start
```

Open **http://localhost:5000**. The first start seeds the default categories, the administrator and two demo students with six months of history, so every chart, budget and tip has real data behind it immediately.

■ Running in development

Two terminals, so the front end hot-reloads:

```
npm run dev          # API on http://localhost:5000
npm run dev:client    # React on http://localhost:5173 (proxies /api to the API)
```

■ Testing every feature

With the server running (`npm start`), in a second terminal:

```
npm run test:e2e --prefix server
```

It registers a throwaway student and works through every functional requirement in the SRS — sign-up, login, password rules, reset, lockout and sessions, profile, categories, income and expenses, recurring entries, CSV import and export, the categoriser learning from corrections, anomaly flags, budgets and alerts, reports and filters, insights, tips, sharing, the chat assistant and every administrator control — then deletes the account. It prints PASS or FAIL for each of its **62 checks**.

■ Reseeding

```
npm run seed
```

Seeding only fills what is missing; it never overwrites existing data. To start completely fresh, delete `server/data/` first.

■ Test data

`server/src/seed/seed.js` runs automatically on first start (and again, safely, on every later one) and writes:

- The twelve default categories — five income (Allowance, Part-time Job, Scholarship, Gift, Other Income) and seven expense (Food, Transport, Hostel/Rent, Academics, Subscriptions, Entertainment, Miscellaneous) — shared by every account.
- The administrator and the two demo students listed in section 03.
- Six months of history per student (`back = 5` down to `0` in the seed loop), income and expenses spread across the categories above with Food deliberately drifting upward in the most recent two months, so the anomaly and trend features have something real to notice on a machine that was only just installed.
- A payment method on every seeded row — cash, bank transfer, card, or one of Easypaisa, JazzCash, SadaPay and NayaPay — weighted per category (rent by bank transfer, a subscription by card, the canteen mostly cash) so the "in account and in cash" figures in section 10 are meaningful from the first login rather than reading a flat 100% cash.



03

User credentials

ROLE	EMAIL	PASSWORD
Administrator	admin@campuscoin.app	Admin@12345
Student	student@campuscoin.app	Student@12345
Student	bilal@campuscoin.app	Student@12345

Students sign in at `/login`. The administrator has a **separate, direct-access sign-in** at `/admin/login`, which posts to its own endpoint and refuses any account that is not an administrator — a student's password is useless there even with the URL.

WHY THESE THREE CANNOT BE LOCKED OUT

The demo accounts are shared, so their passwords are fixed: they cannot be changed, reset or locked out, and anyone evaluating the application can always sign in with the table above. Register a new account to exercise the password features.

04

Passwords and sessions

The SRS asks for secure sessions, hashed passwords, and recovery through a tokenised link. How Campus Coin does each.

STORED AS HASHES

Passwords are hashed with bcrypt (cost 10, a salt per password) and never stored or logged in readable form (`server/src/models/User.js`).

RULES EVERYONE CAN SEE

At least 8 characters, letters and numbers, not one of the most common passwords and not built from the account's own name or email (`server/src/utils/passwords.js`). The sign-up, reset and change forms show the same rules as a live checklist with a strength meter.

RECOVERY BY TOKENISED LINK

"Forgot password" emails a random 64-character token that works once, for one hour; only its SHA-256 hash is stored. The reset page checks the link before showing the form and says so plainly when it has expired. The live site sends real email through Gmail (SMTP with an app password, kept in Vercel's encrypted settings, never in the code), and there the link is **only** ever emailed — showing it on screen would let anyone who knows an address reset that account. On a development machine with no email set up, the link is shown on screen instead so the flow can still be demonstrated. The reply is the same whether or not the email has an account, so the form cannot be used to find accounts.

A PASSWORD ONLY CHANGES THROUGH AN EMAILED LINK

There is no form that sets a new password directly: "Change password" in Settings emails a one-time, one-hour link to the account's own address, and the new password is chosen from that link — so a stolen, still-signed-in session is not enough to take over an account. The old in-place endpoint answers `410 Gone` .

SESSIONS END WHEN THEY SHOULD

Every sign-in token carries the account's session version. Changing or resetting the password raises it, which signs out every other device at once; Settings also has **Sign out everywhere**. A disabled account loses access immediately.

GUESSING IS IMPRACTICAL

Five wrong passwords in a row lock sign-in for 15 minutes; a password reset lifts the lock.

ADMINISTRATOR RESETS

The administrator can issue a temporary password (random, from the crypto module, shown once). It signs the student out everywhere, and every page asks them to choose their own until they do.

TOLD WHEN IT CHANGES

A password change or reset sends a "your password was changed" email, so an account holder notices one they did not make. Welcome, password-link, "password changed" and monthly summary emails share one HTML layout (`server/src/services/emailTemplate.js`): a PNG banner (Gmail does not show SVG), a 600px table with inline styles, one button, and a plain-text version alongside.

05

Configuration

Everything in `server/.env` is optional. The application is designed to run and demonstrate every feature with an empty file.

VARIABLE	WHAT IT DOES WHEN SET	WHAT HAPPENS WHEN IT IS NOT
<code>MONGO_URI</code>	Uses that MongoDB (local or Atlas)	Starts an embedded MongoDB in <code>server/data/db</code>
<code>JWT_SECRET</code>	Signs session tokens	A development default is used
<code>SMTP_HOST</code> , <code>SMTP_PORT</code> , <code>SMTP_USER</code> , <code>SMTP_PASS</code> , <code>MAIL_FROM</code>	Sends password-reset links, "password changed" alerts and shared reports. For Gmail: <code>smtp.gmail.com</code> , port <code>465</code> , the address, and a 16-character app password	On a development machine the reset link is shown on screen instead; on a live deployment nothing is sent and no link is shown
<code>ANTHROPIC_API_KEY</code>	Monthly summaries are rewritten by Claude	Summaries come from the built-in statistical engine
<code>CRON_SECRET</code>	Protects the daily recurring-transaction endpoint	The endpoint refuses all callers; the local server runs the job on a timer anyway

LAYER	WHAT WAS USED, AND WHAT WAS NOT
Front end	React 18 with React Router, built by Vite. No UI framework and no component library: the design system, the layout and all four chart types are written from scratch in CSS and SVG (<code>client/src/styles/</code> , <code>client/src/components/Charts.jsx</code>).
Back end	Node.js with Express 5 and Mongoose.
Database	MongoDB, with an embedded instance as the zero-configuration fallback.
Deployment	Configured for Vercel (<code>vercel.json</code> + <code>api/index.mjs</code>), with the API as a serverless function and the built client served as static files.

07

How the application is put together

```

CampusCoin/
├─ api/index.mjs          Vercel serverless entry point
├─ vercel.json            Build, routing and cron configuration
├─ server/
│  └─ src/
│     ├─ app.js           Express app, routes and error handling
│     ├─ index.js         Local server entry point
│     ├─ config/db.js     Connection, including the embedded fallback
│     ├─ models/          User, Category, Transaction, Budget,
│     │                  Insight, Tip, Notification, Announcement,
│     │                  CategoryHint
│     ├─ middleware/auth.js JWT verification and role gates
│     ├─ routes/          One file per area of the API
│     ├─ services/        The engines – see below
│     └─ seed/            Default categories and demo data
└─ client/
   └─ src/
      ├─ pages/           One file per screen
      ├─ components/      Layout, charts, forms, import wizard
      ├─ context/         Auth, theme and toasts
      ├─ lib/             API client and formatting
      └─ styles/          Design tokens and the stylesheet

```

■ The engines

Everything that makes a judgement lives in `server/src/services/`, separately from the routes that serve it.



FILE	WHAT IT DECIDES
<code>analytics.js</code>	The shared aggregations. Reports, tips, insights and the forecast all read their numbers from here, so they can never disagree with each other.
<code>categorizer.js</code>	Suggests a category from a description.
<code>tips.js</code>	Builds the saving tips and estimates what each is worth.
<code>insights.js</code>	Writes the monthly summary.
<code>anomaly.js</code>	Flags unusually large and duplicate transactions.
<code>forecast.js</code>	Projects next month.
<code>recurring.js</code>	Writes recurring entries when they come due.
<code>alerts.js</code>	Raises budget warnings.
<code>csv.js</code>	Imports and exports transaction files.
<code>chat.js</code>	Works out what a chat question is asking and answers it from the figures above.
<code>llm.js</code>	Optional: rewrites an already-computed summary in warmer language, and answers chat questions the rules did not recognise.

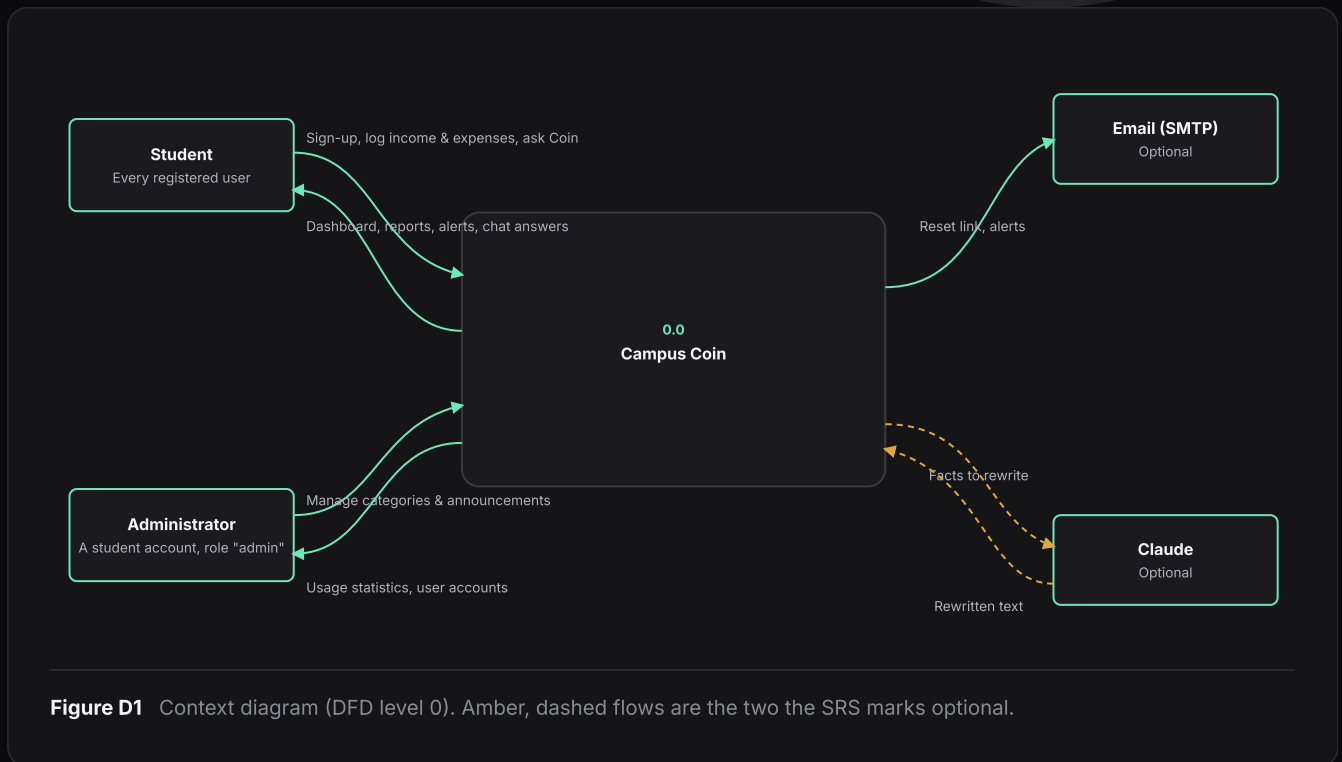
Diagrams and database design

How a request moves through the system, and how the database is shaped - drawn rather than described, so the two can be checked against each other.

■ Context diagram

The whole application as one process, and everyone and everything outside it that it exchanges data with. Two of the four are optional: email only sends where `SMTP_*` is configured, and Claude is only asked where `ANTHROPIC_API_KEY` is set - without either, the equivalent built-in engine answers instead (section 09). Scanning a receipt is deliberately not on this diagram: the photo is read by Tesseract OCR inside the student's own browser and never reaches the server, so it is not a flow this system's boundary has.





■ Level 1: inside the process

The same boundary, opened up into the seven processes a request actually reaches and the five stores they read and write. This is the layer `server/src/services/` is organised at (section 07's table names each engine); the student-facing routes are 2.0, 5.0 and 6.0, and 7.0 is the only one an administrator can reach.

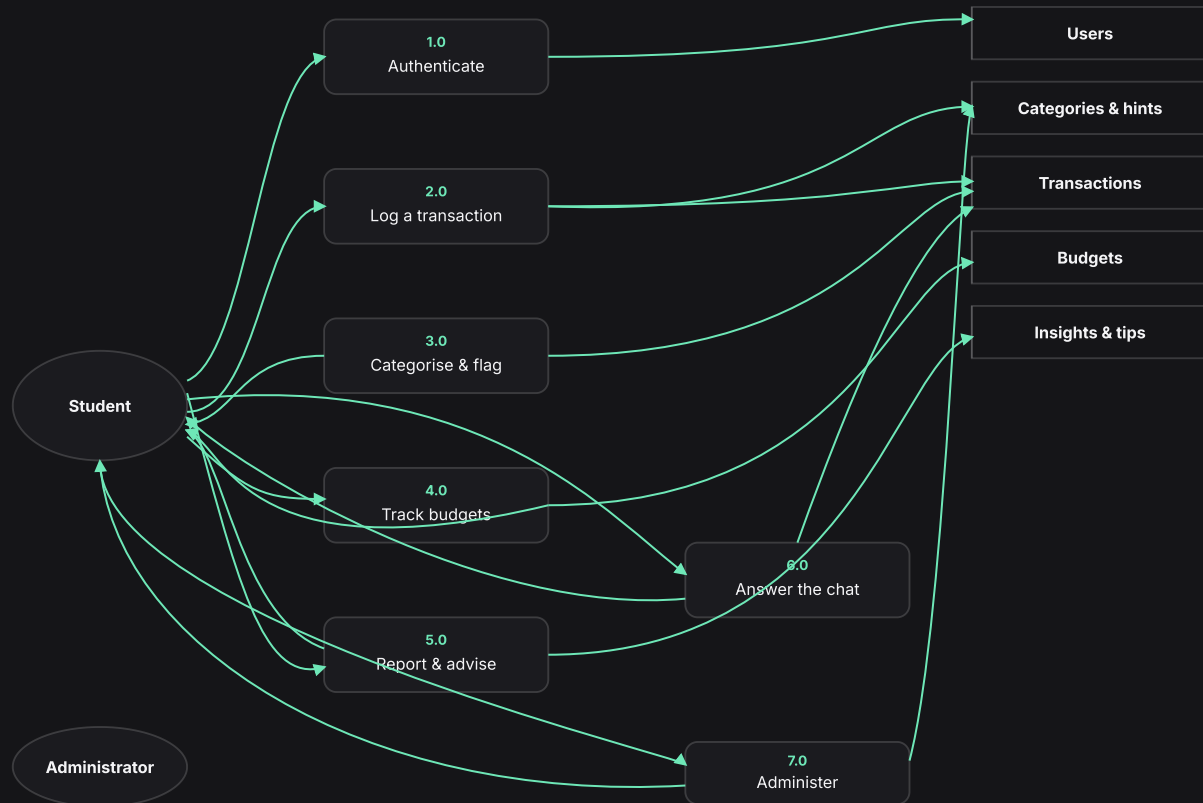


Figure D2 Level 1 DFD. Every arrow into a store is a write, every arrow out is a read; 3.0 both reads and writes Categories & hints, which is how a correction becomes the next suggestion.

■ Flowchart: logging a transaction

The one activity every other feature depends on having real data for, walking through the three judgement calls section 09 explains in detail: the category the assistant suggests, whether the amount looks wrong, and whether it pushes a budget over.

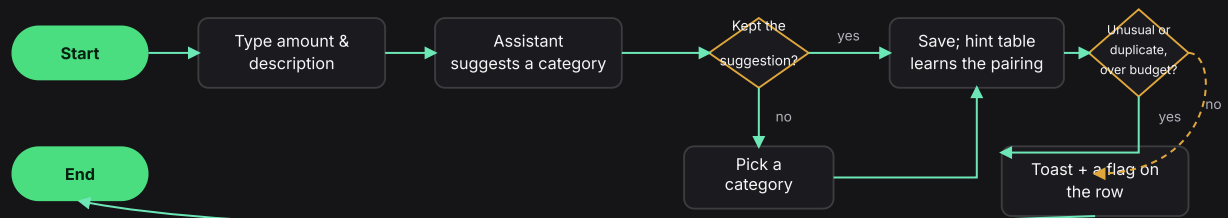


Figure D3 Logging a transaction, from the empty form to a saved row that has already been checked against the student's own budgets and history.

■ Flowchart: forgot password

The tokenised-link recovery section 04 describes, as a flow. The password itself never travels except in this last step, and only over the link the student's own inbox received.

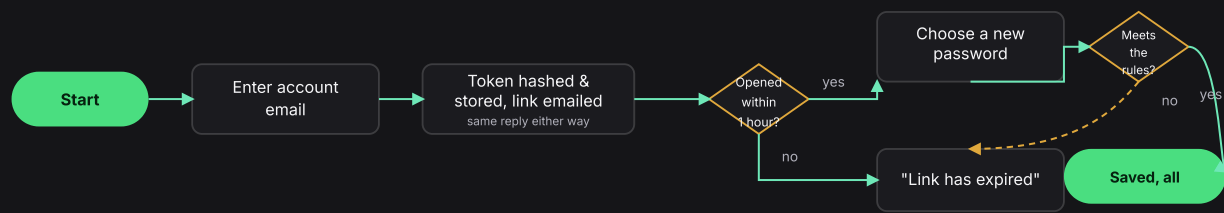
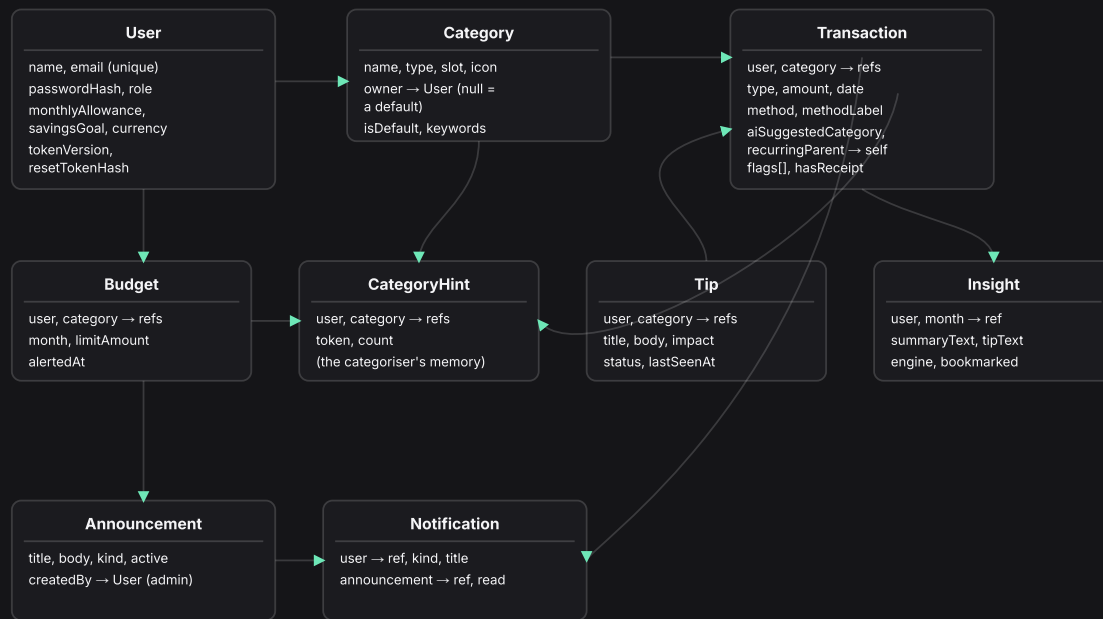


Figure D4 Forgot password. "Saved" also raises the account's session version, which signs every other device out, and sends a "your password was changed" email.



ory to many rows" relationship - the same shape MongoDB's own reference fields describe.

Figure D5 Entity-relationship diagram, from the actual Mongoose schemas, not the SRS's illustrative example.

Database design

MongoDB has no `.sql` script; a Mongoose schema is the equivalent structural definition, and that is what is reproduced below — the real field list from `server/src/models/*.js`, one file per collection, not the SRS's illustrative example. Every reference (`ref`) is an ObjectId pointing at another collection's `_id`, drawn as an arrow in Figure D5 above: `User` is the one everything else points back to; `Category` is shared, since a default category has no owner and every student's own categories sit in the same collection; `Transaction` is the busiest table, with two optional links of its own — `aiSuggestedCategory`, kept so the categoriser can tell whether its guess was accepted, and `recurringParent`, a transaction pointing at the rule that generated it.

USER

One row per account, student or administrator.

FIELD	TYPE	NOTES
<code>name</code>	String	Required.
<code>email</code>	String	Required, unique, lower-cased.
<code>passwordHash</code>	String	bcrypt, cost 10. Never returned by a query unless explicitly selected.
<code>role</code>	enum	<code>student</code> <code>admin</code> .
<code>academicYear</code> , <code>institution</code>	String	Profile fields (SRS 1.6).
<code>monthlyAllowance</code> , <code>savingsGoal</code>	Number	The dashboard's goal ring reads <code>savingsGoal</code> .
<code>currency</code>	enum	PKR, USD, EUR, GBP, INR or AED.
<code>avatar</code> , <code>avatarColor</code>	String	An optional photo as a small data URL; a coloured-initial fallback otherwise.
<code>preferences</code>	Object	<code>theme</code> , <code>fontScale</code> , <code>reducedMotion</code> , <code>alertsEnabled</code> — the accessibility settings.
<code>tokenVersion</code>	Number	Bumped on a password change or "sign out everywhere"; every older session token stops working.
<code>resetTokenHash</code> , <code>resetTokenExpires</code>	String, Date	Only the hash of a reset token is stored. Not selected by default.
<code>failedLogins</code> , <code>lockUntil</code>	Number, Date	Sign-in lockout after repeated wrong passwords. Not selected by default.
<code>disabled</code> , <code>mustChangePassword</code>	Boolean	Set by an administrator.

CATEGORY

A system default (`owner: null`) or one student's own.

FIELD	TYPE	NOTES
<code>name</code> , <code>type</code>	String, enum	<code>income</code> <code>expense</code> . Unique together with <code>owner</code> .
<code>owner</code>	ObjectId → User	<code>null</code> for the twelve defaults, seeded once and shared by every account.
<code>isDefault</code> , <code>archived</code>	Boolean	Archived categories keep their history but drop out of the add-transaction list.
<code>icon</code>	String	Which Fluent Emoji SVG represents this category.
<code>slot</code>	Number 1–7	A position in the validated categorical palette, not a raw colour, so light and dark mode each pick their own correct shade.
<code>keywords</code>	[String]	Seed words for the categorisation assistant, refined by <code>CategoryHint</code> below.

TRANSACTION

Every logged rupee, income or expense.



FIELD	TYPE	NOTES
<code>user</code> , <code>category</code>	ObjectId → User, Category	Both required.
<code>type</code> , <code>amount</code>	enum, Number	<code>income</code> <code>expense</code> ; amount must be greater than zero.
<code>description</code> , <code>note</code>	String	What the student typed; <code>description</code> is also what the categorisation assistant reads.
<code>date</code> , <code>month</code>	Date	<code>month</code> is <code>date</code> rounded down to the 1st, written automatically before every save, so reports group by month without a date pipeline.
<code>method</code> , <code>methodLabel</code>	enum, String	cash, bank, easypaisa, jazzcash, sadapay, nayapay, card, or other (named in <code>methodLabel</code>) — where the money actually moved.
<code>aiSuggestedCategory</code> , <code>aiAccepted</code>	ObjectId, Boolean	What the assistant proposed, and whether the student kept it — the correction that trains <code>CategoryHint</code> .
<code>recurring</code>	Object	<code>enabled</code> , <code>frequency</code> (weekly/monthly/yearly), <code>nextRun</code> , <code>endsOn</code> — the rule, held on the parent row.
<code>recurringParent</code>	ObjectId → Transaction	Set on every row a rule generated, pointing back at that rule.
<code>source</code>	enum	manual csv recurring.
<code>receipt</code> , <code>hasReceipt</code>	String, Boolean	The photo is excluded from every list query by default (<code>select: false</code>) and fetched only when a single transaction is opened.
<code>flags</code>	[String]	<code>duplicate</code> and/or <code>large</code> , written by the anomaly check.

BUDGET

One spending cap per category per month.

FIELD	TYPE	NOTES
<code>user</code> , <code>category</code> , <code>month</code>	ObjectId, ObjectId, Date	Unique together — one cap per category per month.
<code>limitAmount</code>	Number	≥ 0 .
<code>alertedAt</code>	Number	The highest threshold (80 or 100) already warned about, so a student hears once per level rather than on every transaction.

INSIGHT

The stored monthly summary Coin writes (SRS 1.6, "stores insight history").

FIELD	TYPE	NOTES
<code>user</code> , <code>month</code>	ObjectId, Date	Unique together — one insight per student per month.
<code>summaryText</code> , <code>tipText</code>	String	The narrative, and the one piece of advice tied to it.
<code>highlights</code>	[Object]	Categories that moved against the student's own trend: name, amount, previous amount, change%, direction.
<code>stats</code>	Object	income, expense, balance, savingsRate, topCategory, transactionCount — a frozen copy of the figures the summary was written from.
<code>engine</code>	enum	<code>statistical</code> (the built-in writer) or <code>llm</code> (Claude rewrote it) — see section 09.
<code>bookmarked</code>	Boolean	SRS 1.6, "bookmarks a saving tip or monthly insight".

TIP, CATEGORYHINT, NOTIFICATION, ANNOUNCEMENT

The four smaller collections behind the tips engine, the categorisation assistant, the bell menu and admin broadcasts.

COLLECTION	KEY FIELDS	PURPOSE
<code>Tip</code>	<code>user</code> , <code>key</code> , <code>title</code> , <code>body</code> , <code>impact</code> , <code>status</code>	One saving tip. <code>key</code> identifies the same piece of advice across regenerations, so a pin or a dismissal (<code>status: pinned dismissed</code>) survives the tips engine running again.
<code>CategoryHint</code>	<code>user</code> , <code>token</code> , <code>category</code> , <code>count</code>	A per-student word-to-category frequency table. Every save adds to it; a correction reinforces a different pairing — the memory behind the categorisation assistant learning from that student alone.
<code>Notification</code>	<code>user</code> , <code>kind</code> , <code>title</code> , <code>body</code> , <code>link</code> , <code>read</code>	Budget warnings, anomaly flags and admin announcements, in one bell menu. <code>kind</code> is one of budget-warning, budget-exceeded, anomaly, insight or announcement.
<code>Announcement</code>	<code>title</code> , <code>body</code> , <code>kind</code> , <code>active</code> , <code>createdBy</code>	Written by an administrator (SRS 1.6, "system-wide announcement or tip templates"); shown on every student dashboard while <code>active</code> .

09

The parts that make judgements

The SRS asks for AI assistance in two places, and for a chatbot. They are separate features, and a language model is only ever an optional finishing step: every figure is calculated by Campus Coin itself.

CATEGORISING A DESCRIPTION

Runs entirely on Campus Coin's own server. Each word of a description is counted against the category the student actually chose, building a per-student word-to-category frequency table (`CategoryHint`). Correcting a suggestion is what teaches it: the correction records a different pairing than the one that was proposed, so the next guess leans the other way. A new account starts from seed keywords on the default categories until it has a history of its own. Every suggestion shows its confidence and its reason, and can always be overridden.

WRITING THE MONTHLY SUMMARY

Computes every figure from the database first, then — only if `ANTHROPIC_API_KEY` is set — asks Claude to rewrite those already-computed facts in friendlier language. The model is never asked to do arithmetic, so a summary cannot contradict the report it came from. Without a key the built-in statistical writer produces the summary instead, and the feature reports which one wrote it.

THE CHAT ASSISTANT

The SRS's "AI ChatBot" lives in the bubble in the corner of every student page; what it has learned, and a button to make it forget, are in Settings. It is built in rather than embedded from tawk.to or Tidio, because a third-party widget cannot see the student's data. A rule-based router in `chat.js` recognises the kind of question being asked (balance, one category, the category breakdown, budgets, the forecast, the largest expense, saving tips, "can I afford 2,500?", "which category is chai at the canteen?", this month against last) and answers from the same aggregations the reports use. Only a question no rule recognises is passed to Claude, and only if a key is set, together with a sheet of already-computed facts and an instruction to use nothing else. The conversation is kept in the browser tab and is never stored on the server.



This is Coin. The same disc as the logo, with eyes — so the mark on the tab and the face answering a question are visibly the same thing, not two different pieces of clip art.

SCANNING A RECEIPT

The SRS's optional OCR. In the add-transaction form a student takes or drops a photo of a receipt (or tries one of the two samples in `client/public/samples`). The photo is shrunk in the browser and read with Tesseract OCR, also in the browser — no API key and nothing is uploaded to be read. Plain rules in `client/src/lib/receipt.js` then pick out the amount paid (preferring "grand total", "net payable" and "total" lines and ignoring subtotal, tax, cash and change), the shop and the date, fill the form, and let the categoriser suggest the category. The student checks everything before saving.

THE SAVING TIPS

Not AI at all. Each rule compares this month against the student's own three-month average and states how much money the advice is worth per month; the list is ranked by that figure. No tip is generic — every one quotes real numbers from that student's history.

THE FORECAST

A least-squares line through the last six months. It refuses to answer with fewer than three months of data and reports how far the line typically sits from the actual points, rather than implying precision it does not have.

STATED PLAINLY IN THE INTERFACE

None of this is financial advice, and the interface says so where it matters.

Design and accessibility

The interface is written rather than assembled. This section is also the SRS's **design specification** — the decisions behind the interface, named rather than left to be inferred from the screens in section 13.

LIGHT BY DEFAULT, WITH A DARK THEME

Both are designed against their own surface rather than one being an inversion of the other. The choice is saved to the device *and* the account, so it follows the student to another machine. Settings also offers "match my device", which follows the operating system live.

BLACK, WHITE AND ONE GREEN

The application sits in a black frame with a slim rail of round icon buttons that slides open on hover to show each page's name; the content is one white panel with large rounded corners, buttons are black pills, and one green (#22c55e) carries the figures. Categories take shades of green and grey. Headings and figures are set in **Geist**, cut for a dashboard rather than adapted from a text face; body copy and tables are **Inter**, for its legibility in a row of transactions. The styles live in `client/src/styles/design9.css`; the code and all the copy are our own.

REAL SVG PICTURES

Every category has its own object — a burger for Food, a bus for Transport, a house for Hostel/Rent — drawn from Microsoft's **Fluent Emoji** in the flat style (MIT licence). Only the 49 SVGs the application uses are copied into `client/src/assets/emoji`, with the licence beside them, so nothing is fetched from outside. Menu and button icons stay as clean line icons (`Icon.jsx`), drawn on one 24px grid following the Lucide conventions.

REAL BRAND LOGOS

When a transaction names a company — "Netflix share", "Foodpanda dinner", "Spotify Premium", "Daraz", "JazzCash", "PTCL" — the list shows that company's logo on its brand colour instead of the category picture (`client/src/lib/brands.js`). The logos come from **Simple Icons** (CC0), bundled with the application. Careem and inDrive are not in that set, so they get a tile in their brand colour with their initial rather than an imitation logo. The logos only identify the merchant; Campus Coin is not affiliated with them.

CHARTS THAT READ ON A PHONE

Money in against money out is drawn as paired bars built from HTML (`MonthBars` in `DashCharts.jsx`), so the scale, months and figures stay full size on any screen instead of shrinking with an SVG. Tapping a month shows its three figures. The trend lines on the report cards are labelled with their first and last month.

THE DASHBOARD, AGAINST SRS 1.6 "PERSONALIZED DASHBOARD"

A personal greeting; the month's balance as money in against money out; quick-add buttons (chai, rickshaw, printing, allowance) that open the form already filled in; saving tips and Coin's insight built from the student's own history; and the two named widgets — **This Month's Top Category** (its picture, amount and share) and **Budget vs. Actual** (each budget's cap against what was spent, with a status). Categories has a **Manage Own Categories** section for adding, editing and deleting personal income and expense categories beside the defaults.

CASH AND ACCOUNT, NOT JUST A TOTAL

Every transaction also records where the money moved — cash, a bank transfer, a card, or a named wallet (Easypaisa, JazzCash, SadaPay, NayaPay), with a typed name and its own colour for anything else. The **In account and in cash** dashboard card splits the month both ways: what came in to an account against what came in as notes, and the reverse for spending, each expandable into the individual places the money actually went. The calendar and the Transactions filter answer the same question at the day and the row: a "Paid with" filter, and a mark beside every entry. Coin understands it too — "how much cash did I spend", "how much came in through Easypaisa" — through the same aggregation the dashboard reads, so the two can never disagree. It is deliberately not a running balance: the application never sees a transfer between two of a student's own places, so a true "cash in hand" figure would drift wrong the first time an ATM withdrawal was spent as notes. Reporting the two flows honestly, rather than guessing a number it cannot back up, is the more defensible choice — and the more informative one, since most students are paid digitally and spend the difference as cash.

MOTION EXPLAINS, IT DOES NOT DECORATE

Sections arrive top to bottom, charts draw themselves (the donut sweeps in ranked order, lines trace, bars grow), headline figures count up, and budget bars fill — the SRS's "smooth transitions while charts and insights are generated". Every entrance ends at the element's normal resting style, so with the system's reduced-motion setting on, pages simply appear complete.

ACCESSIBILITY

Text size is adjustable from Settings and scales the whole interface, charts included. Keyboard focus is always visible, `prefers-reduced-motion` is respected, the layout works down to 360px, and every chart is backed by a table of the same numbers.



11

Deploying to Vercel

1. Push the repository to GitHub and import it into Vercel.
2. Add a MongoDB Atlas connection string as `MONGO_URI` (or connect the Atlas integration, which provides `MONGODB_URI`). **This is required** — the embedded database cannot run in a serverless function.
3. Set `JWT_SECRET` and `CRON_SECRET`.
4. Deploy. `vercel.json` handles the build, the API routing and the daily cron that writes recurring transactions.

12

Deliverables checklist

- ☒ Problem definition — section 01
- ☒ Design specification — section 10
- ☒ Diagrams: flowcharts, context and level-1 DFDs — section 08
- ☒ Database design, every collection's fields — section 08, and the schema files themselves (`server/src/models/*.js`) as the ".sql or schema files" the SRS asks for
- ☒ Test data — section 02, and seeded automatically on first run (`server/src/seed/`)
- ☒ Installation instructions (mandatory) — section 02
- ☒ User credentials for all user types (mandatory) — section 03
- ☒ AI tools used, acknowledged — section 01
- ☒ Sitemap on the home page — `/` and `/sitemap`
- ☒ No source code in this document
- ☐ Demonstration video (.mp4) — to record

13

The interface

The screens a student meets, in the order they meet them.

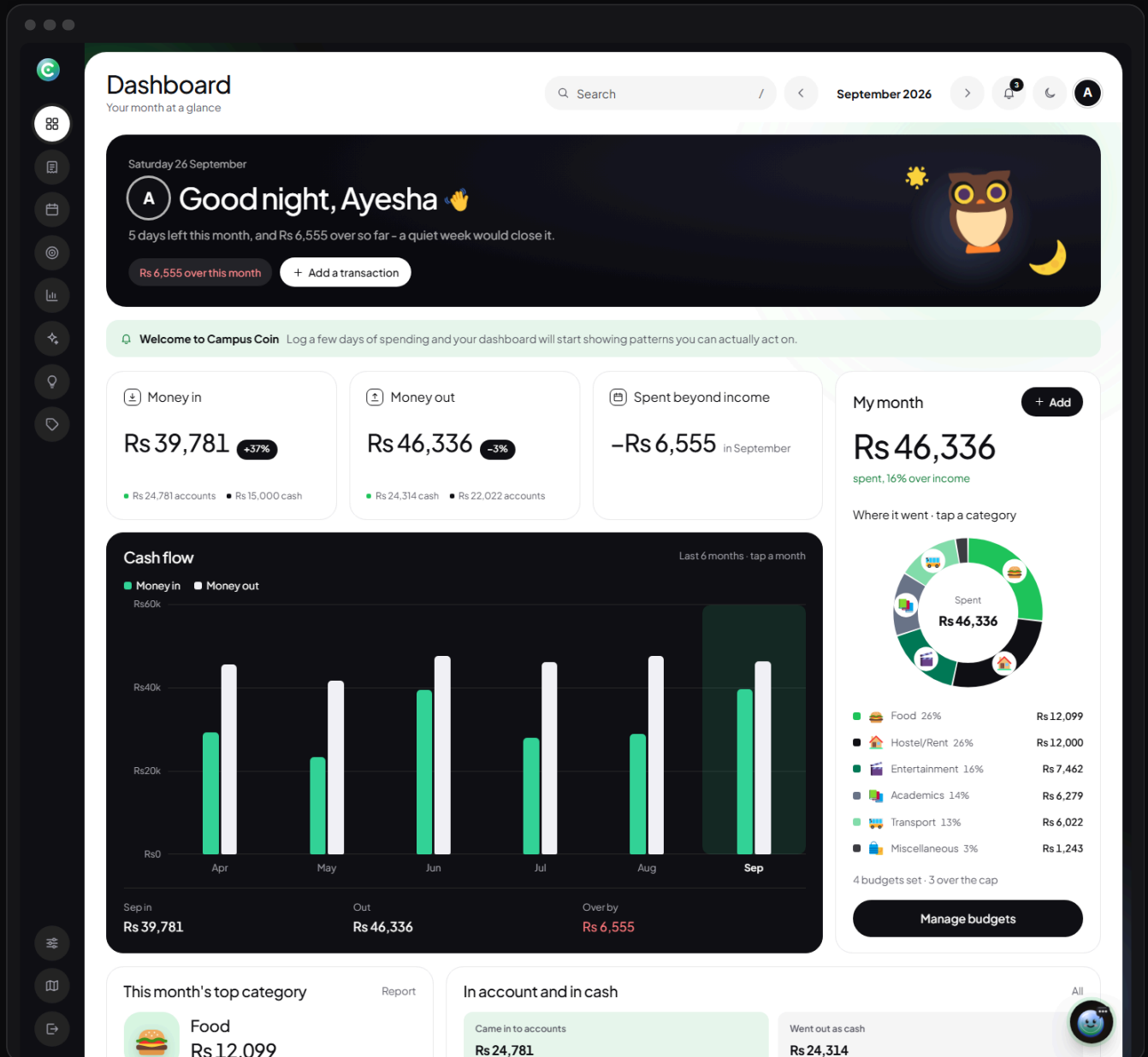


Figure 1 The dashboard: the month's figures, the cash flow card, the spending calendar and the latest transactions.

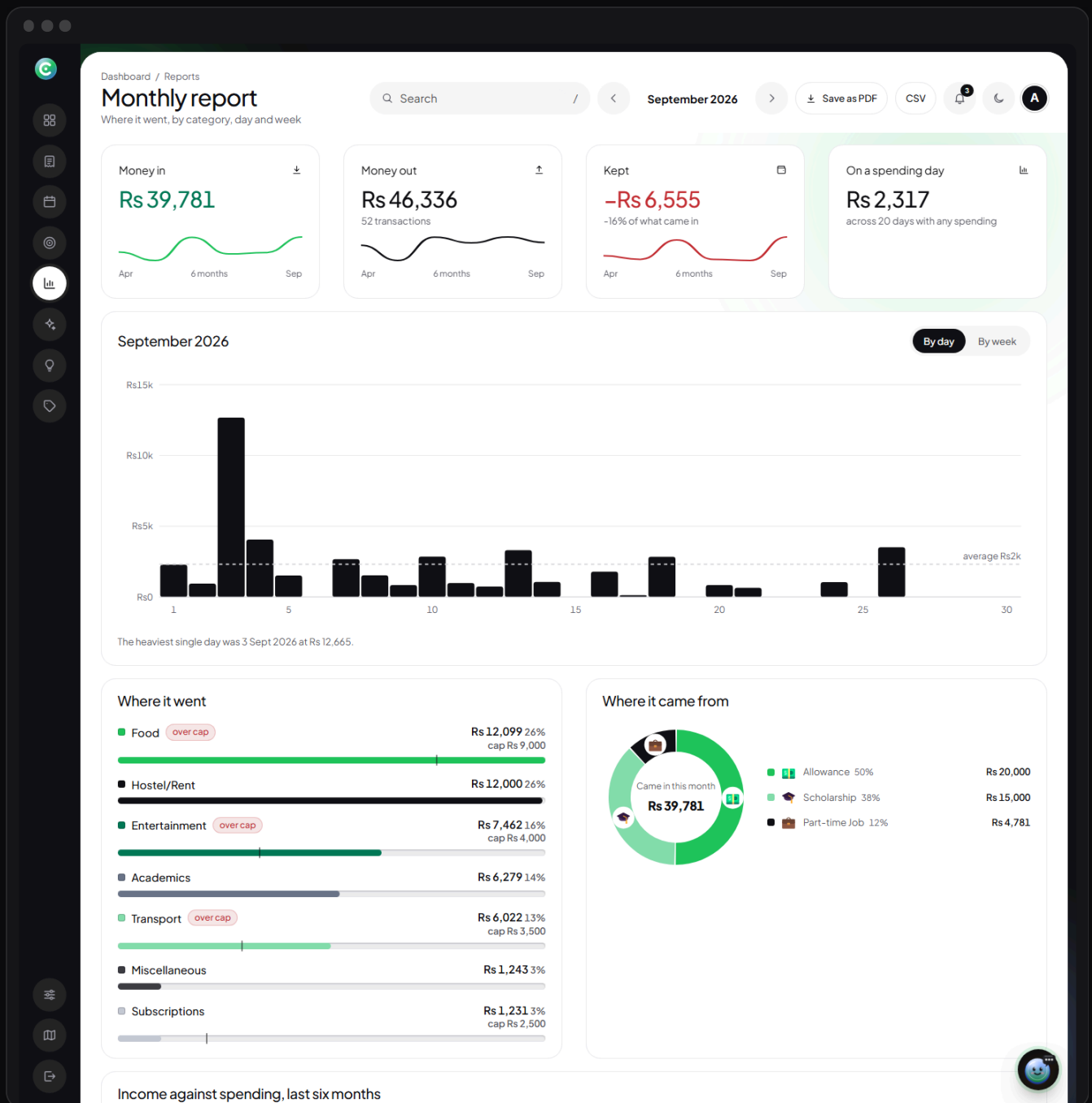


Figure 2 Reports: six months side by side, the category breakdown, and the trend cards.

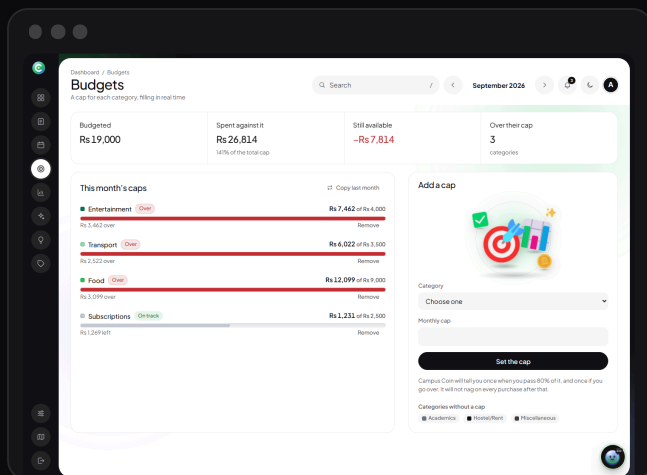


Figure 3 Budgets, each cap against what was spent.

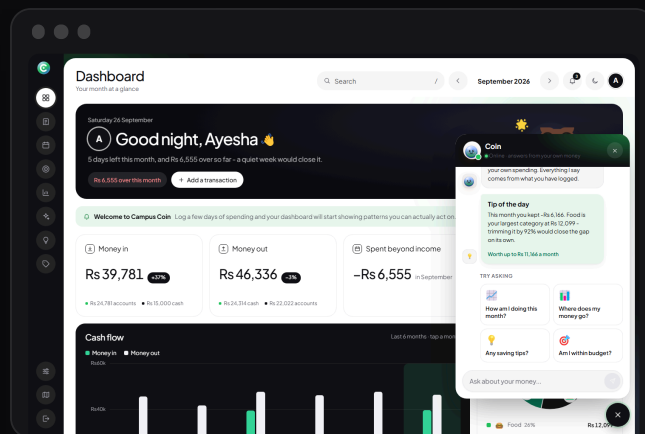


Figure 4 Coin, answering from the student's own figures.



Figure 5 A receipt read in the browser by Tesseract OCR.

Figure 6 The same interface at 390px.



That's Campus Coin.

Every figure in this report came from the student's own numbers, the same way every figure Coin gives back in the app does — never invented, never guessed, and never financial advice.

campuscoin-sable.vercel.app

github.com/hamzaharoonk07-cloud/campuscoin

CAMPUS COIN · PROJECT REPORT

